

Cryptography and Network Security Exam

Kevine Kabarira
Student ID: 4202670038

September 28, 2026

1 Introduction

This project assesses security risks in a polytechnic institute environment and demonstrates basic cryptography, file integrity checking, and network security controls. Sample data was used throughout the project. No real student records, passwords, or encryption keys were uploaded to the repository.

2 Risk Assessment

2.1 Assets, Vulnerabilities and Consequences

- **Student records server:** Guest network access could allow unauthorised access to student records.
- **Staff accounts:** Weak passwords could lead to account compromise.
- **File-transfer system:** Unencrypted transfers could allow data interception.

2.2 Risk Ranking

- **Guest access:** Likelihood - High. Impact - High. Guests may reach sensitive records.
- **Weak passwords:** Likelihood - High. Impact - High. Weak passwords can be guessed or compromised.
- **Unencrypted transfers:** Likelihood - Medium. Impact - High. Data can be intercepted during transfer.

2.3 Recommended Controls

- Guest access: Use firewall rules and network segmentation.
- Weak passwords: Enforce a strong password policy.
- Unencrypted transfers: Use SFTP.

3 Python Security Toolkit

A Python security toolkit was developed to demonstrate encryption, decryption, hashing, integrity verification, and error handling.

3.1 Encryption

The toolkit uses Fernet symmetric encryption from the Python `cryptography` library. A sample student record was encrypted and saved as an encrypted output file.

The encryption key was stored outside the GitHub repository to prevent exposure.

3.2 Decryption and Verification

The encrypted file was successfully decrypted. The SHA-256 hash of the original file was compared with the SHA-256 hash of the decrypted file.

Original SHA-256:

```
9072923fe0a1c93d2fe2f0f564281c78c0d6e7cb9ddb1a315578fcbaf428b877
```

Decrypted SHA-256:

```
9072923fe0a1c93d2fe2f0f564281c78c0d6e7cb9ddb1a315578fcbaf428b877
```

Contents match: True

Integrity check: PASS

The matching hashes confirm that the decrypted file has the same contents as the original sample file.

3.3 Integrity Detection

A copy of the sample file was modified after the original hash was calculated.

Original hash:

```
9072923fe0a1c93d2fe2f0f564281c78c0d6e7cb9ddb1a315578fcbaf428b877
```

Modified file hash:

```
50d29235c461aec22174106942a54ea16096fbb5a55aacdb52ba74150cdb9f
```

Modified file detected: True

This demonstrates that SHA-256 can detect changes to the file.

3.4 Error Handling

The toolkit was tested with a missing input file. The test correctly raised a `FileNotFoundError`.

4 Testing

Python's built-in `unittest` framework was used.

The tests included:

1. SHA-256 hash verification.
2. Password strength checking.
3. Missing-file error handling.
4. Encryption and decryption execution.
5. Integrity verification.

The missing-file test produced:

```
Ran 1 test
OK
```

The main toolkit execution produced:

```
File encrypted successfully.
File decrypted successfully.
Contents match: True
Integrity check: PASS
```

5 Network Traffic Filtering

Network traffic filtering was tested in an authorised laboratory environment using UFW.

The Ubuntu laboratory server used IP address 192.168.56.104. The Windows laboratory client used IP address 192.168.56.1.

A Python HTTP server was used on TCP port 8080.

```
python3 -m http.server 8080 --bind 0.0.0.0
```

The firewall used a default deny policy for incoming traffic and allowed outgoing traffic.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
```

For the permitted test, TCP port 8080 was temporarily allowed:

```
sudo ufw allow 8080/tcp
```

The Windows client tested the connection:

```
Test-NetConnection 192.168.56.104 -Port 8080
```

Result:

```
TcpTestSucceeded : True
```

The open rule was then removed:

```
sudo ufw delete allow 8080/tcp
```

An explicit deny rule was added:

```
sudo ufw deny from 192.168.56.1 to any port 8080 proto tcp
```

The first blocked test was:

```
Test-NetConnection 192.168.56.104 -Port 8080
```

Result:

```
TcpTestSucceeded : False
```

A second blocked test checked port 8081:

```
Test-NetConnection 192.168.56.104 -Port 8081
```

Result:

```
TcpTestSucceeded : False
```

The tests therefore demonstrated one permitted TCP connection and two blocked TCP connections.

Detailed firewall commands and test results are documented in `filter_tests.md`.

6 GitHub Repository

The complete project is maintained in the GitHub repository:

<https://github.com/kevine-do/cryptography-network-security-exam>

The repository contains the source code, tests, risk assessment, firewall evidence, documentation, and technical report.

7 Conclusion

The project identified security risks and recommended suitable controls. A Python security toolkit was developed to encrypt and decrypt sample student data, calculate SHA-256 hashes, detect file modification, and handle missing files.

The tests demonstrated successful encryption, decryption, integrity verification, and error handling. Network traffic filtering was also tested in an authorised laboratory environment using UFW.

The project demonstrates practical applications of cryptography, file integrity checking, access control, and network security.